JOB PORTAL MANAGMENT SYSTEM

School of Computing

Bachelor of Technology
in
Computer Science & Engineering

By

Sk.Mansoor (VTU26005)

10211CS224
Full Stack Application Development
Slot : E2-B18

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
SCHOOL OF COMPUTING

VEL TECH RANGARAJAN Dr.SAGUNTHALA R&D INSTITUTE OF SCIENCE AND TECHNOLOGY
(Deemed to be University Estd u/s 3 of UGC Act, 1956)
CHENNAI 600 062, TAMILNADU, INDIA

May, 2026

BONAFIDE CERTIFICATE

It is certified that the work contained in the Full Stack Application Development report titled ”Job Portal Management System” by Sk.Mansoor (VTU26005) has been carried out in Winter semester of Academic year 2025-2026(WS2526).

Signature of Faculty

Dr. Hemamalini .S

Assistant Professor

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr.Sagunthala R&D

Institute of Science and Technology

April, 2026

Signature of Course Co-ordinator

Dr. Sumithra

Assistant Professor(Senior Grade)

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr.Sagunthala R&D

Institute of Science and Technology

April, 2026

DECLARATION

I declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. I also declare that i have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

Sk.Mansoor

Date:06/05/2026

ABSTRACT

The Job Portal API is a robust and scalable backend system designed to streamline the recruitment process by connecting job seekers and employers through a centralized digital platform. The primary objective of this project is to provide an efficient, secure, and user-friendly environment where users can search for jobs, apply for positions, and manage recruitment workflows seamlessly. The system is developed using PHP for backend processing and MySQL as the relational database, ensuring efficient data handling and reliable performance. The application follows a RESTful architecture, enabling smooth communication between the client and server through standard HTTP methods and JSON data formats. Core functionalities of the system include user registration, secure login, job posting, job search with filtering options, and application management. These features form the foundation of a complete CRUD-based job management system. To enhance the system beyond basic functionality, several advanced features have been integrated. Third-party API integration is implemented to enable email notifications and real-time communication, ensuring that users receive updates regarding job applications and

account activities. An OTP (One-Time Password) verification system is incorporated to strengthen user authentication and prevent unauthorized access, thereby improving overall security. Furthermore, a chatbot module is included to provide instant support and guidance to users. The chatbot assists users in navigating the platform, answering frequently asked questions, and simplifying the job application process. This feature enhances user engagement and reduces dependency on manual support systems. In addition, map integration is utilized to visually represent job locations, allowing users to better understand geographical aspects such as proximity and accessibility. This improves decision-making for job seekers by providing spatial context to job listings. Security is a key focus of the system. Advanced security mechanisms such as password hashing, token-based authentication using JSON Web Tokens (JWT), and role-based access control are implemented to ensure data protection and secure API interactions. These measures safeguard sensitive user information and prevent unauthorized operations within the system.

Keywords: Job Portal API, RESTful Architecture, PHP, MySQL, CRUD Operations, JSON, Firebase Authentication, OTP Verification, Third-Party API Integration, Chatbot, Maps Integration, JWT Authentication, Role-Based Access Control, Web Application Security

LIST OF FIGURES

LIST OF TABLES

LIST OF ACRONYMS AND ABBREVIATIONS

API	Application Programming Interface
HTTP	HyperText Transfer Protocol
REST	Representational State Transfer
JSON	JavaScript Object Notation
CRUD	Create, Read, Update, Delete
DB	Database
SQL	Structured Query Language
RDBMS	Relational Database Management System
PDO	PHP Data Objects
JWT	JSON Web Token
CORS	Cross-Origin Resource Sharing

TABLE OF CONTENTS

Chapter 1

INTRODUCTION

1.1 Introduction

In the modern digital era, the process of job searching and recruitment has significantly evolved from traditional methods to online platforms. Organizations and job seekers increasingly rely on web-based systems to simplify hiring processes, reduce time consumption, and improve accessibility [1]. A Job Portal system serves as a centralized platform that connects employers and job seekers, enabling efficient communication and streamlined recruitment workflows. The Job Portal API project is designed to develop a secure, scalable, and efficient backend system that facilitates job-related operations such as user registration, authentication, job posting, job searching, and application management. The system is built using PHP as the backend programming language and MySQL as the relational database, ensuring structured data storage and reliable performance. The application

follows a RESTful architecture, allowing seamless communication between client and server using standard HTTP methods and JSON data exchange [2]. To enhance the system beyond basic functionalities, modern technologies and advanced features have been incorporated. Firebase Authentication is integrated to provide secure and reliable user authentication, including OTP-based verification, ensuring that only authorized users can access the system. This significantly improves security and reduces the risk of unauthorized access. Additionally, third-party API integrations are utilized to enable functionalities such as email notifications and external data handling, making the system more dynamic and interactive. A chatbot feature is implemented to assist users by providing instant responses to queries, guiding them through the platform, and improving overall user experience. This reduces dependency on manual support and ensures 24/7 availability. The integration of map services further enhances the usability of the system by allowing users to view job locations visually. This helps job seekers understand geographical aspects such as distance, accessibility, and location relevance, enabling better decision-making.[4] Security is a major focus of the system. Advanced mechanisms such as password encryption, token-based authentication using JSON Web Tokens (JWT), and role-based access control are implemented to protect user data and ensure secure API interactions.

1.2 Aim of the Project

The primary aim of the Job Portal API project is to design and develop a secure, scalable, and efficient backend system that facilitates seamless interaction between job seekers and employers through a centralized digital platform. The project focuses on simplifying the recruitment process by providing functionalities such as user registration, authentication, job posting, job searching, and application management [3]. Another key objective is to enhance the system with modern technologies and advanced features to meet real-world requirements. This includes the integration of Firebase Authentication for secure login and OTP-based verification, ensuring reliable user identity management. The project also aims to incorporate third-party APIs to enable dynamic functionalities such as email notifications and external data integration. Furthermore, the project seeks to improve user experience by implementing intelligent features such as a chatbot for instant user support and map integration to visually represent job locations. These features help users navigate the platform efficiently and make informed decisions.[4] Security is a major focus of this project. The aim is to implement strong security mechanisms such as password encryption, JSON Web Token (JWT) based authentication, and role-based access control to protect sensitive user data.

1.3 Scope of the Project

The scope of the Job Portal API project is to design and develop a comprehensive backend system that facilitates efficient interaction between job seekers and employers through a centralized online platform. The system is intended to support essential recruitment functionalities such as user registration, secure authentication, job posting, job searching, and application management [3]. The project focuses on building a RESTful API that enables seamless communication between the frontend interface and the backend server using standard HTTP methods and JSON data formats[5]. It provides role-based access for different types of users, such as job seekers and administrators, ensuring proper control over system operations and data access. The scope also includes the integration of advanced technologies to enhance functionality and user experience. Firebase Authentication is incorporated to provide secure login and OTP-based verification, ensuring reliable user identity management. Third-party APIs are utilized to enable features such as email notifications and external service integration, making the system more dynamic and interactive [5]. Additionally, the project includes a chatbot module that assists users by answering queries, guiding them through the platform, and improving overall usability. Map integration is also included to visually display

job locations, helping users understand geographical aspects and make informed decisions. From a security perspective, the project covers the implementation of essential measures such as password encryption, token-based authentication using JSON Web Tokens (JWT), and role-based access control. These features ensure the protection of user data and prevent unauthorized access to the system. The system is designed to handle moderate to large volumes of data efficiently through optimized database operations, indexing, and pagination techniques. This ensures scalability and consistent performance as the number of users and job listings increases [6]. However, the scope of the project is limited to backend development and API functionalities. Frontend implementation, mobile application development, and deployment to large-scale production environments are considered beyond the current scope but can be implemented as future enhancements. In conclusion, the project aims to deliver a feature-rich, secure, and scalable backend solution for modern job recruitment systems, with the flexibility to extend and integrate additional functionalities in the future The system is designed with a modular architecture, allowing easy integration of future technologies such as AI-based job recommendations and resume analysis. Furthermore, the API structure supports cross-platform compatibility, enabling seamless integration with web and mobile applications.

1.4 Framework

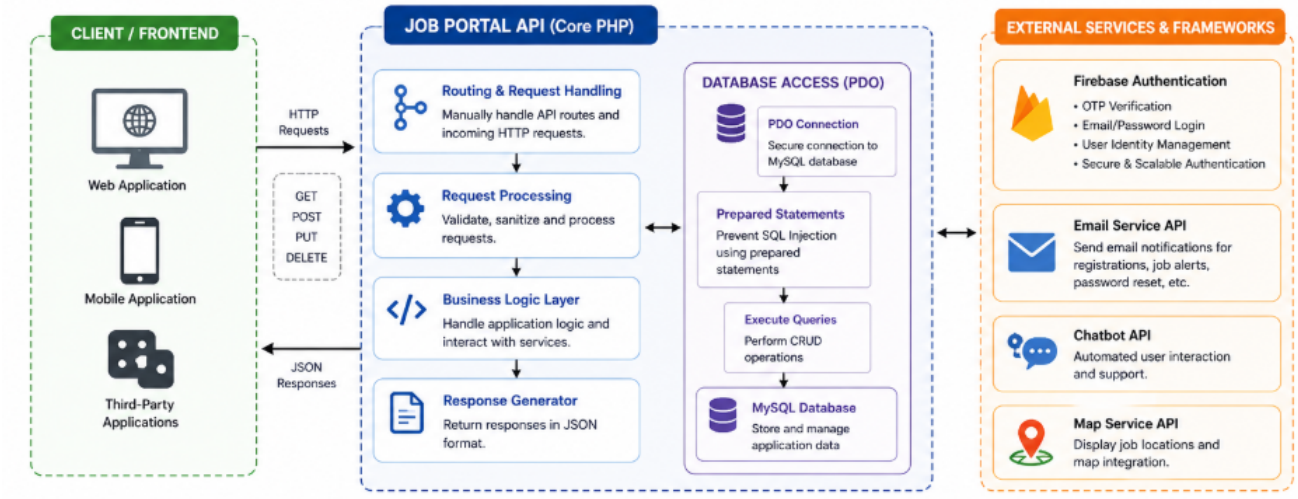

Figure 1.1: Framework of job portal system

Figure 1.1 illustrates The Job Portal system. it is developed using a lightweight and flexible approach without relying on a full-stack backend framework. Instead, the system is built using core PHP, which provides greater control over the application logic and helps in understanding the fundamental concepts of backend development. This approach allows the developer to manually handle routing, request processing, and database interactions, making the system highly customizable [7]. For authentication and user management, Firebase Authentication is integrated as a cloud-based framework. It provides secure and scalable authentication services, including OTP-based verification, email/password login, and user identity management. Firebase simplifies the implementation of authentication mechanisms while ensuring high security standards. The project follows the RESTful API

architectural style, which acts as a framework for designing network-based applications. It uses standard HTTP methods such as GET, POST, PUT, and DELETE to perform operations and communicates using JSON format. This ensures compatibility with various frontend technologies and platforms [8]. Additionally, third-party APIs are used as supporting frameworks to extend system functionality. These include email service APIs for notifications, chatbot integration frameworks for automated user interaction, and map service APIs for displaying job locations. These integrations enhance the overall capability of the system without increasing complexity in the core backend [6]. For database interaction, PHP Data Objects (PDO) is used as a database abstraction framework. It provides a consistent and secure way to connect to the MySQL database, execute queries, and prevent SQL injection attacks through prepared statements [7]. In summary, the project combines core PHP development with modern cloud-based and third-party frameworks such as Firebase Authentication, RESTful architecture, and API integrations to build a secure, scalable, and feature-rich backend system.

Chapter 2

SURVEY OF SIMILAR APPLICATION IN MARKET

In today's digital era, several online job portals are widely used by job seekers and recruiters to simplify the hiring process. These platforms provide various features such as job search, resume upload, employer interaction, and application tracking. A study of these existing systems helps in understanding their functionalities, strengths, and limitations, which can be used to design an improved solution [7]. One of the most popular professional networking platforms is LinkedIn. It allows users to build professional profiles, connect with industry experts, and apply for jobs directly. The platform is widely used for networking and is especially effective for mid-level and senior job roles. It also provides features such as skill endorsements, job recommendations, and recruiter communication tools [8]. Another widely used job portal is Indeed, which focuses primarily on job discovery and ap-

plication. It provides a simple and user-friendly interface where users can search for jobs using filters such as location, salary, and experience level. It also supports resume uploads and job alerts, making it easier for candidates to find suitable opportunities quickly. Naukri.com is one of the leading job portals in India, offering a large database of job listings across multiple industries. It provides personalized job recommendations based on user profiles and allows recruiters to directly contact candidates. It is particularly useful for entry-level and mid-level job seekers in the Indian market. In addition to these, platforms like Apna focus on providing quick job opportunities, especially for freshers and blue-collar roles. It enables direct communication between job seekers and recruiters, making the hiring process faster and more efficient [9]. Despite the availability of these platforms, there are certain limitations. Many job portals lack integrated features such as secure OTP-based authentication, chatbot support for instant assistance, and map-based job visualization. Additionally, some platforms may include fake job listings or lack proper verification mechanisms, which can affect user trust. The proposed Job Portal API aims to overcome these limitations by integrating advanced features such as Firebase Authentication with OTP verification, chatbot assistance, map integration for job locations, and enhanced security mechanisms [7].

Chapter 3

FRAMEWORK DESCRIPTION

3.1 Frontend Implementation

The frontend of the Job Portal system is designed to provide an interactive and user-friendly interface for job seekers and administrators. It is developed using basic web technologies such as HTML, CSS, and JavaScript, ensuring responsive design and smooth user experience[2]. The frontend communicates with the backend through RESTful APIs using HTTP requests and displays data dynamically. Firebase Authentication is integrated to handle secure login and OTP verification. Key features include user registration, login, job search, job application, chatbot interaction, and map-based job location display [12]. Overall, the frontend ensures easy navigation, real-time interaction, and efficient communication with the backend system.The frontend is designed to ensure smooth navigation and responsive performance across different devices and screen sizes [13].

3.1.1 Environment Setup

The environment for the Job Portal system is set up using modern development tools to ensure efficient development and testing. A code editor such as Visual Studio Code is used for writing and managing the project files, while web browsers are used for testing and debugging the application. For backend connectivity, a local server environment such as XAMPP or WAMP is configured to run PHP and manage the MySQL database [12]. All required dependencies and libraries are properly installed and configured. Firebase is integrated to handle authentication features such as secure login and OTP verification. API endpoints are configured to enable smooth communication between frontend and backend systems. Additionally, tools like Postman are used to test API requests and responses, ensuring that the system functions correctly and efficiently.Proper configuration ensures smooth development and testing process.The development environment is organized to ensure smooth workflow and efficient project management. All necessary software tools and dependencies are properly installed and configured before execution. Version control practices can be used to manage changes and maintain code consistency. Proper testing environments are maintained to identify and fix errors during development. This setup helps in achieving reliable performance and simplifies future maintenance and upgrades.

3.1.2 Structure of the Project

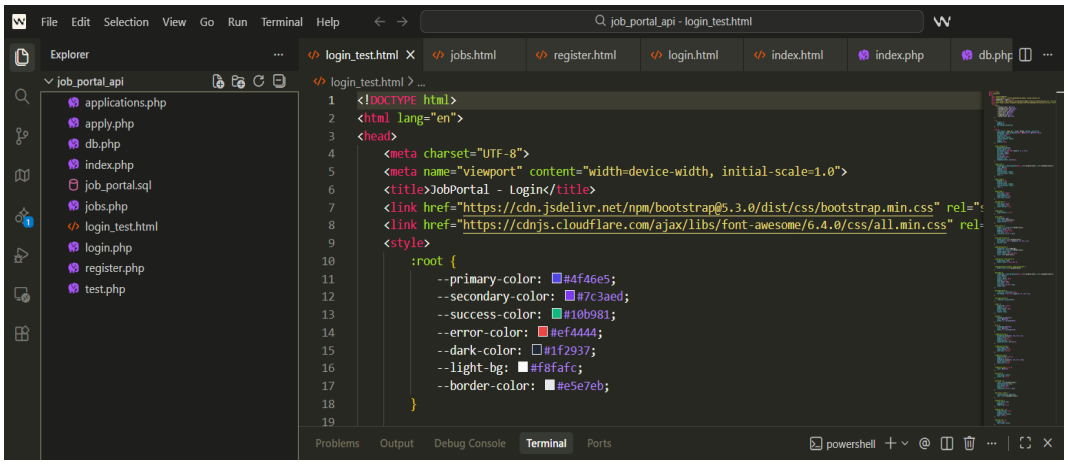

Figure 3.1: Structure of the job portal system

Figure 3.1 illustrates that project follows a modular and well-organized structure to ensure easy development, maintenance, and scalability. The system is divided into separate components such as frontend, backend, and database, each handling specific functionalities. The frontend manages user interaction and interface design, while the backend processes requests, handles business logic, and communicates with the database. The backend is structured into different modules such as authentication, job management, and application handling. Each module is responsible for specific operations like user login, job posting, job search, and application submission. Firebase Authentication is integrated within the authentication module to provide secure login and OTP verification. The database structure includes tables for users, jobs, and applications, with proper relationships to maintain data integrity.

3.1.3 Execution Procedure

- Users access the application by registering and logging in through Firebase Authentication with OTP verification.
- After successful authentication, users can perform various operations such as:
  - Searching for jobs
  - Viewing job details
  - Applying for job positions
- The frontend communicates with the backend by sending requests through RESTful APIs using HTTP methods (GET, POST, PUT, DELETE).
- The backend processes incoming requests, executes business logic, and interacts with the database using PDO.
- The database stores and retrieves data related to users, jobs, and applications.
- The backend sends responses back to the frontend in JSON format.
- The system is tested using tools like Postman to ensure API functionality and correct request-response handling.

3.2 Backend Implementation

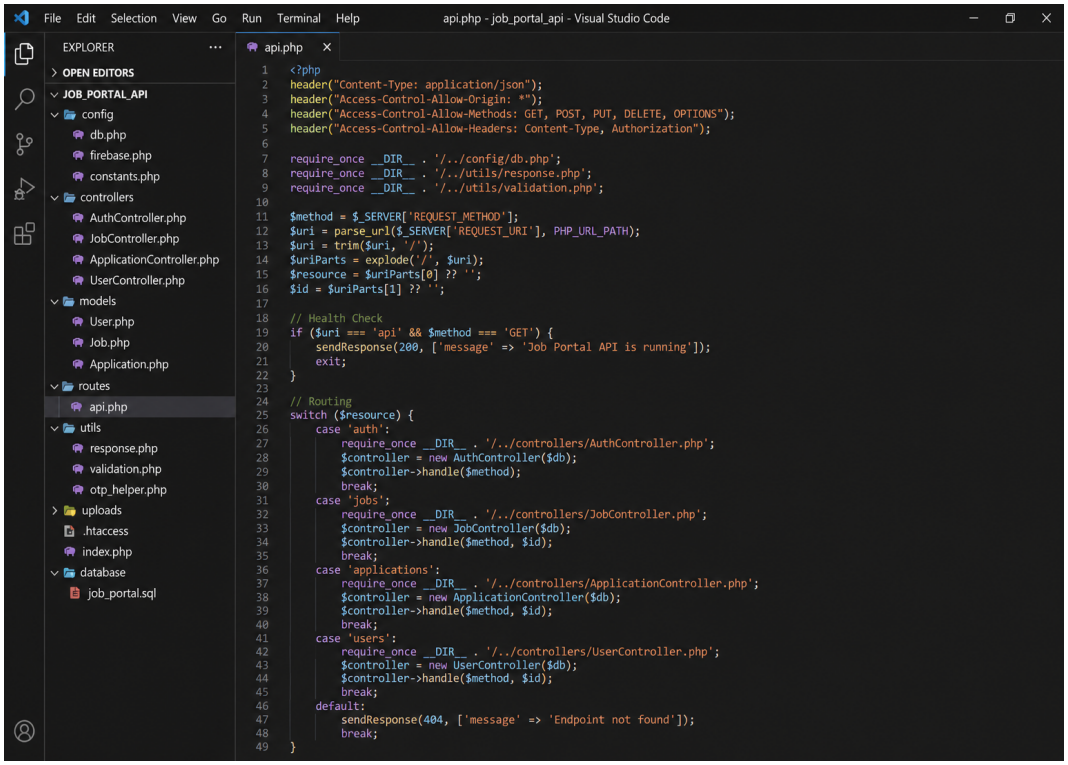

Figure 3.2: Bckend Implementation of the job portal system

Figure 3.2 illustrates The backend of the Job Portal system is developed using PHP and follows a RESTful API architecture to handle client requests efficiently. It is responsible for processing user actions such as registration, login, job posting, job searching, and application submission. The backend communicates with the MySQL database using PHP Data Objects (PDO), ensuring secure and efficient data handling. Firebase Authentication is integrated into the backend to provide secure user authentication with OTP verification. Additionally, JSON Web Tokens (JWT) are used to manage user sessions and protect API endpoints from unauthorized access. Role-based access

control is implemented to differentiate between user and admin functionalities. The backend is structured into modular components such as authentication, job management, and application handling, making the system easy to maintain and extend. Third-party APIs are also integrated to enable features like email notifications and chatbot support. Overall, the backend ensures secure, scalable, and efficient operation of the system.The backend is designed to handle multiple user requests efficiently with proper request routing and response handling mechanisms. Input validation and data sanitization are implemented to ensure data integrity and prevent security vulnerabilities such as SQL injection. Error handling techniques are used to provide meaningful responses and maintain system stability. The use of optimized queries and indexing improves database performance and reduces response time. This structured backend design supports scalability and allows easy integration of future enhancements.The backend ensures efficient communication between different system components through well-defined API endpoints. Logging mechanisms can be implemented to monitor system activities and debug issues effectively. This approach improves reliability and helps maintain consistent system performance.The backend is implemented using Core PHP in a modular structure with controllers, models, routes, and utility files, handling RESTful API requests.

3.2.1 Environment Setup

The backend environment for the Job Portal system is configured using PHP as the server-side scripting language and MySQL as the database management system. A local server environment such as XAMPP or WAMP is used to run the application, providing support for Apache, PHP, and database services. This setup enables efficient development, testing, and execution of backend functionalities. All necessary configurations, including database connection settings and API endpoint setup, are properly defined. Firebase Admin SDK is integrated to support secure authentication and OTP verification. Required libraries and dependencies are installed to ensure smooth operation of all modules. Tools such as Postman are used to test API endpoints and verify request-response cycles. The environment is structured to support debugging, error handling, and performance monitoring, ensuring reliable and efficient system operation.The environment is configured to support seamless integration between frontend and backend components. Proper file structure and configuration management are maintained to ensure smooth deployment. Backup and recovery mechanisms can be considered to prevent data loss during development. Regular testing is performed to verify system stability in different environments. This setup ensures flexibility and supports future upgrades and scalability.

3.2.2 Structure of the Project

The structure of the project is designed in a layered and modular approach to ensure smooth communication between different components. The system consists of the frontend (user interface), backend (RESTful API), authentication module using Firebase, and the database (MySQL). Each component is connected through well-defined interfaces and data flow mechanisms. The frontend interacts with the backend by sending HTTP requests, and the backend processes these requests using business logic. Firebase Authentication is used to verify user identity through secure login and OTP verification. The backend further communicates with the database to store and retrieve data such as user details, job listings, and applications. Additional components such as chatbot integration and map services are also connected to the backend to enhance user experience. The overall structure ensures scalability, security, and efficient data handling within the system.The project structure follows a clear flow of data from the user interface to the backend and then to the database, ensuring efficient processing of requests. Each layer is designed to perform specific tasks, reducing complexity and improving system organization. The use of APIs ensures smooth communication between components and allows easy integration with external services.

3.2.3 Execution Procedure

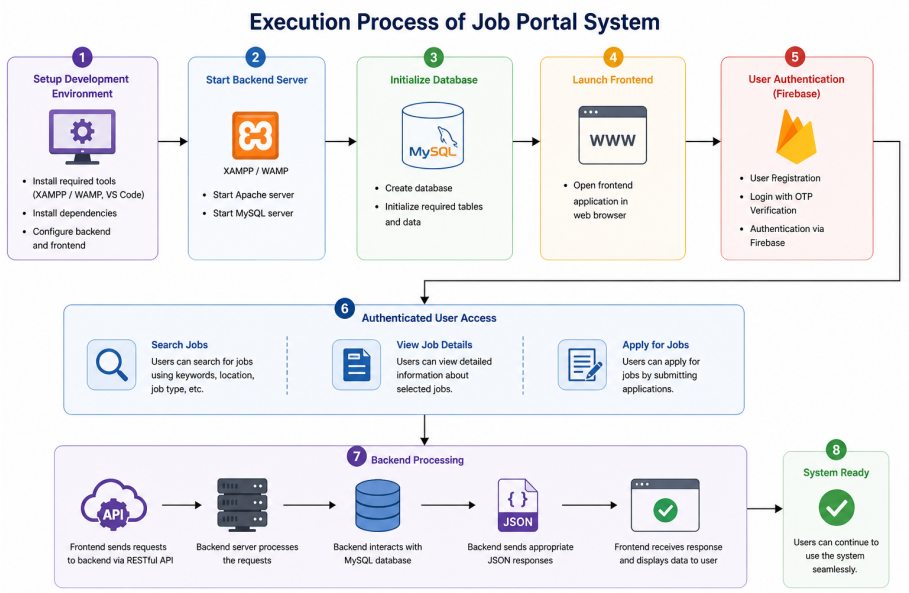

Figure 3.3: Execution process of job portal system

Figure 3.3 llustrates The execution of the Job Portal system begins with setting up the required development environment and ensuring that all necessary tools, dependencies, and configurations are properly installed. The backend server is started using a local server environment such as XAMPP or WAMP, and the MySQL database is initialized with the required tables and data. The frontend application is then launched in a web browser to interact with the system. Users can access the platform by registering and logging in through Firebase Authentication with OTP verification. Once authenticated, users can perform various operations such as searching for jobs, viewing job details, and applying for positions.

3.3 Database Implementation

3.3.1 Database Configuration

The database for the Job Portal system is configured using MySQL to ensure reliable and efficient data storage. A dedicated database is created, and necessary tables such as users, jobs, and applications are defined with appropriate fields and data types. Primary keys are assigned to uniquely identify each record, and foreign keys are used to establish relationships between tables. The connection between the backend and the database is established using PHP Data Objects (PDO), which provides a secure and flexible way to interact with the database. Configuration details such as database name, username, password, and host are properly set in the connection file. Security measures such as prepared statements and input validation are implemented to prevent SQL injection and ensure safe data transactions. Indexing is applied to important columns to improve query performance. The database is tested thoroughly to ensure correct data storage, retrieval, and update operations, making the system reliable and efficient.The database configuration is designed to support efficient data management and quick access to frequently used information. Backup mechanisms can be implemented to prevent data loss and ensure recovery in case of system failure.

3.1.2 Schema Structure

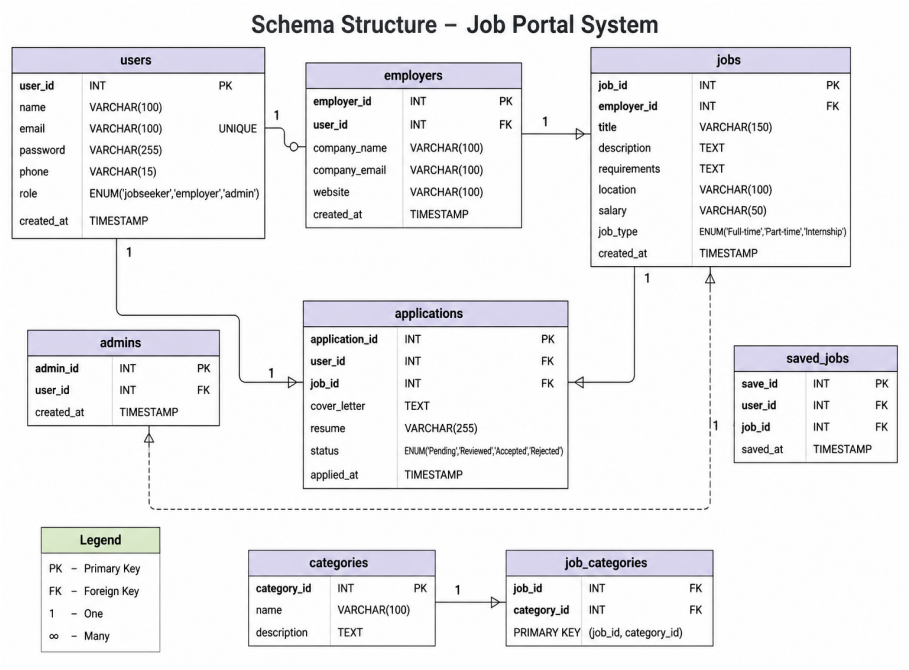

Figure 3.4: Schema Structure of the job portal

Figure 3.4 illustrates The schema structure of the Job Portal system represents the organization of data within the database using multiple interconnected tables. The primary table is the users table, which stores user details such as user ID, name, email, password, phone number, and role (job seeker, employer, or admin). This table acts as the central entity for authentication and user management. The jobs table stores information related to job postings, including job ID, title, description, location, salary, and job type. Each job is linked to an employer through a foreign key relationship. The applications table connects users and jobs, storing details about job applications such

as application ID, user ID, job ID, resume, status, and application date.Additional tables such as admins and saved jobs provide extended functionality. The admins table manages administrative users, while the saved jobs table allows users to bookmark jobs for future reference. The schema also includes categories and job categories tables to classify jobs into different categories, enabling better organization and filtering.The schema is designed following normalization principles to minimize data redundancy and ensure consistency across all tables. Each entity is clearly defined with appropriate attributes, making the database easy to understand and manage. Constraints such as primary keys, foreign keys, and unique fields are applied to maintain data accuracy and enforce relationships.

Tables created

Table 3.1: Users Table

Field	**Type**	**Description**
user_id	INT (PK)	Unique user ID
name	VARCHAR(100)	User name
email	VARCHAR(100)	User email (unique)
password	VARCHAR(255)	Encrypted password
phone	VARCHAR(15)	Contact number
role	VARCHAR(20)	User role
created_at	TIMESTAMP	Account creation time

Table 3.1 illustrates the structure of the User table, which stores essential information about registered users such as user ID, name, email, encrypted password, phone number, role, and account creation time.

Table 3.2: Jobs Table

Field	Type	Description
job_id	INT (PK)	Unique job ID
title	VARCHAR(150)	Job title
description	TEXT	Job description
location	VARCHAR(100)	Job location
salary	VARCHAR(50)	Salary details
job_type	VARCHAR(50)	Job type
created_at	TIMESTAMP	Job posted date

Table 3.2 illustrates that the Jobs table stores essential information related to job postings, including job ID, title, description, location, salary, job type, and the date the job was created, enabling efficient management and retrieval of job data in the system.

Table 3.3: Applications Table

Field	Type	Description
application_id	INT (PK)	Unique application ID
user_id	INT (FK)	Reference to user
job_id	INT (FK)	Reference to job
resume	VARCHAR(255)	Resume file path
status	VARCHAR(50)	Application status
applied_at	TIMESTAMP	Application date

Table 3.3 illustrates that the Applications table is structured to maintain records of job applications by linking users and jobs through foreign keys, while also storing resume paths, application status, and timestamps, ensuring proper tracking, relationship mapping, and processing of applications in the job portal system.

Table 3.4: Saved Jobs Table

Field	**Type**	**Description**
save_id	INT (PK)	Unique ID
user_id	INT (FK)	Reference to user
job_id	INT (FK)	Reference to job
saved_at	TIMESTAMP	Saved date

Table 3.4 illustrates that the Saved Jobs table stores information about jobs bookmarked by users, linking users and jobs through foreign keys while recording the date when a job was saved, thereby enabling users to easily access and manage their preferred job listings.

Chapter 4

METHODOLOGIES

4.1 Project Architecture

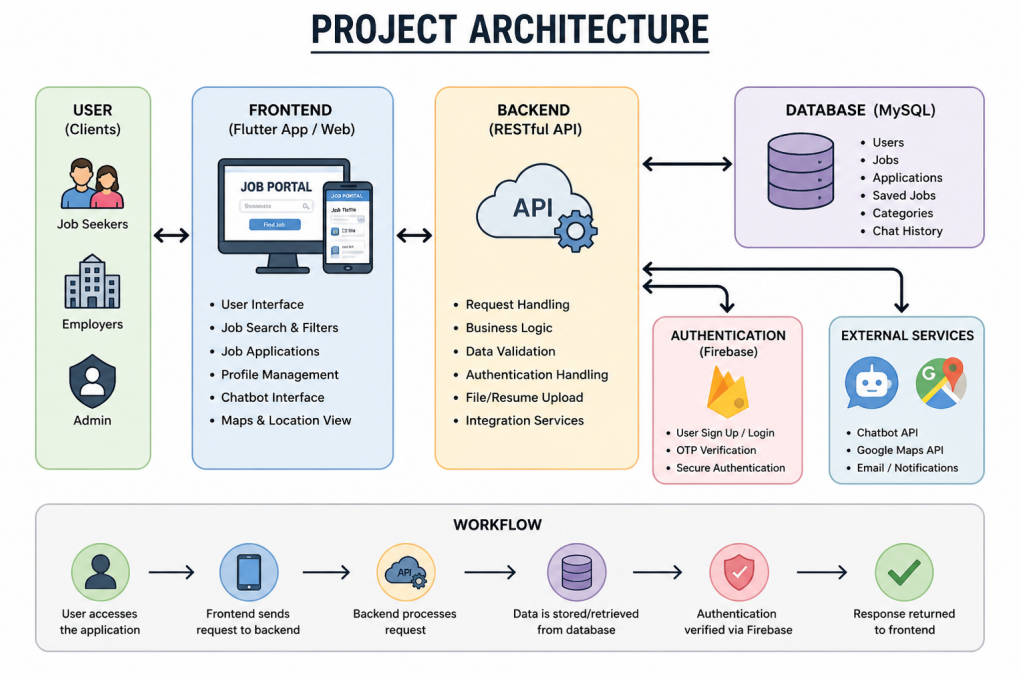

Figure 4.1: Project Architecture of Job Portal System

Figure 4.1 illustrates The architecture of the Job Portal system follows a layered and modular design to ensure scalability, security, and efficient data flow. The system is divided into major components including the frontend (user interface), backend (RESTful API), authen-

tication module using Firebase, and the database (MySQL). Each layer is responsible for specific functionalities and communicates with other layers through well-defined interfaces. The frontend layer handles user interaction and sends requests to the backend through HTTP protocols. The backend processes these requests using business logic and interacts with the database to store and retrieve data. Firebase Authentication is integrated to manage secure user login and OTP verification, ensuring that only authorized users can access the system. Additional services such as chatbot integration and map APIs are connected to the backend to enhance user experience and provide real-time assistance and location-based information. The architecture supports smooth communication between all components, ensuring reliability and performance. Overall, the project architecture is designed to be flexible and scalable, allowing easy integration of new features and handling increasing numbers of users efficiently.The above architecture diagram illustrates the interaction between different components of the Job Portal system. It clearly shows how the frontend communicates with the backend through RESTful APIs, and how data is processed and stored in the database. The integration of Firebase Authentication ensures secure user verification through OTP and login mechanisms.

4.2 DataFlow Diagram

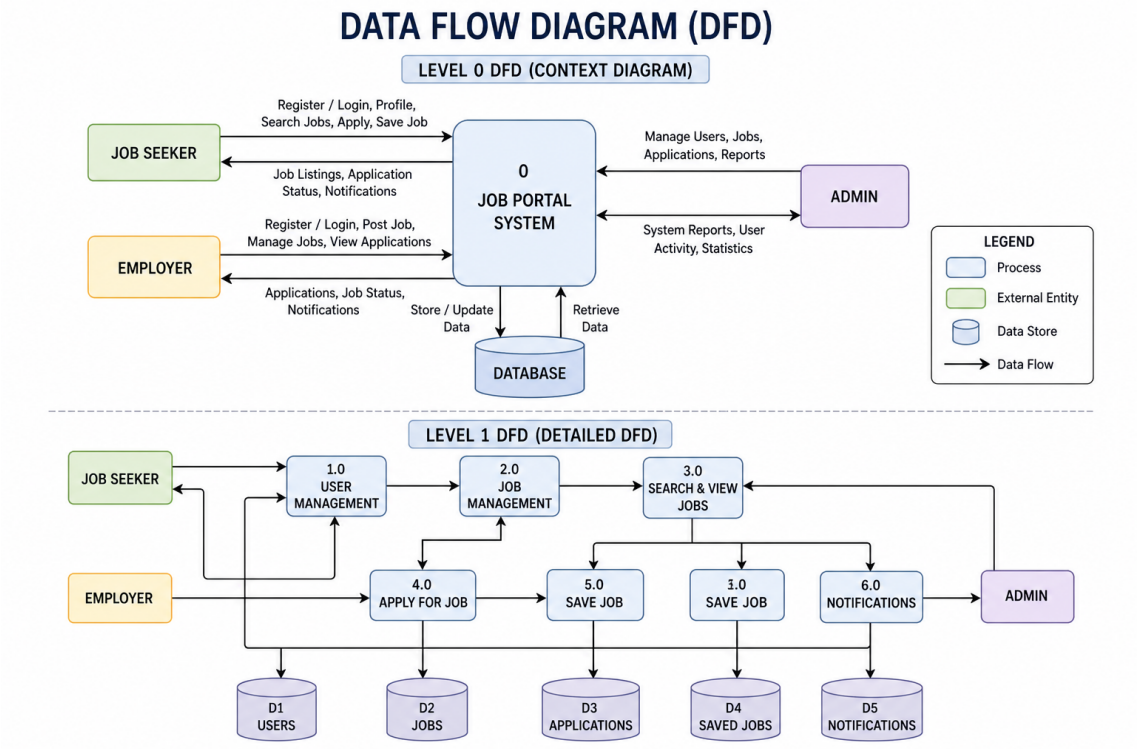

Figure 4.2: Data Flow Diagram (DFD) of Job Portal System

Figure 4.2 illustrates The Data Flow Diagram (DFD) represents how data moves within the Job Portal system. It shows the interaction between external entities such as Job Seeker, Employer, and Admin with the system. The Level 0 DFD provides a high-level overview of the entire system. The Level 1 DFD gives a detailed view of processes like user management, job management, and applications. Data is stored and retrieved from the database through different processes. Overall, the DFD helps in understanding the system workflow and data processing clearly.The DFD clearly illustrates the flow of data between different components of the system.

4.3 Module 1:

This module focuses on understanding the overall requirements of the Job Portal system and designing a structured approach for its implementation. It begins with analyzing the problem domain, where the need for an efficient and centralized job recruitment platform is identified. The system aims to connect job seekers and employers through a secure and user-friendly interface, reducing the complexity of traditional hiring processes. In this phase, both functional and non-functional requirements are clearly defined. Functional requirements include user registration, login, job posting, job searching, application submission, and job management. Non-functional requirements include system performance, scalability, security, usability, and reliability. Special attention is given to security requirements such as Firebase Authentication, OTP verification, and role-based access control to ensure safe handling of user data.

The system design is then developed based on these requirements. A layered architecture is planned, consisting of frontend, backend, authentication services, and database components. The frontend is responsible for user interaction, while the backend handles business logic and communication with the database. RESTful APIs are designed to enable smooth data exchange between components. Database

design is an important part of this module. The schema is created with tables such as users, jobs, applications, and saved jobs. Proper relationships are established using primary and foreign keys to maintain data integrity. Normalization techniques are applied to reduce redundancy and improve efficiency. Additionally, system diagrams such as Data Flow Diagrams (DFD), architecture diagrams, and schema diagrams are prepared to visualize the system structure and data flow. These diagrams help in better understanding and communication of the system design. Feasibility analysis is also conducted to evaluate the technical, economic, and operational viability of the project. The system is designed to be scalable so that future features like AI-based job recommendations, resume analysis, and real-time communication can be integrated easily. Overall, this module provides a strong foundation for the project by clearly defining requirements and designing a structured, secure, and scalable system architecture. It ensures that the development process proceeds smoothly and the final system meets real-world expectations.Proper documentation is maintained throughout this phase to guide future development and maintenance. This module plays a critical role in ensuring the success and reliability of the entire system.

4.4 Module 2:

This module focuses on the development and implementation of the core backend functionalities of the Job Portal system. It involves building a robust and scalable server-side architecture that handles user requests, processes data, and communicates with the database efficiently. The backend is developed using PHP and follows a RESTful API approach to ensure smooth interaction between the frontend and backend components. In this phase, essential features such as user registration, login, job posting, job searching, and job application are implemented. Each functionality is developed as a separate API endpoint, allowing modular and organized handling of different operations. The system supports role-based access, where job seekers can search and apply for jobs, while administrators can manage job listings and user activities. Firebase Authentication is integrated to provide secure user login and OTP-based verification. This ensures that only authorized users can access the system. Additionally, JSON Web Tokens (JWT) are used to manage user sessions and protect API endpoints from unauthorized access. These security mechanisms play a crucial role in safeguarding user data and maintaining system integrity. The backend is connected to a MySQL database using PHP Data Objects (PDO), which ensures secure and efficient database operations. Prepared statements

are used to prevent SQL injection attacks, and proper validation is implemented for all user inputs. The system supports operations such as inserting user data, retrieving job listings, updating job details, and managing applications. Third-party APIs are integrated in this module to enhance system functionality. Email APIs are used to send notifications related to job applications and account activities. Chatbot APIs are integrated to provide automated user assistance, and map APIs are used to display job locations, improving user experience. Performance optimization techniques such as indexing, pagination, and optimized queries are implemented to ensure fast data retrieval and efficient handling of large datasets. Error handling and logging mechanisms are also included to identify and resolve issues effectively. Overall, this module builds the core working system of the Job Portal application by combining backend logic, database management, security features, and API integrations. It ensures that the system is reliable, secure, and capable of handling real-world requirements efficiently. The modular design of the backend allows easy maintenance and future enhancements. This module ensures smooth integration of all components and provides a strong foundation for advanced features in the system.

4.5 Module 3:

This module focuses on integrating advanced features into the Job Portal system to enhance functionality, security, and user experience. After implementing the core backend operations in previous modules, this phase adds intelligent and real-world capabilities that make the system more interactive, secure, and scalable. One of the key features implemented in this module is Firebase Authentication, which provides secure user login and OTP-based verification. This ensures that only authorized users can access the system and significantly reduces the risk of fake accounts and unauthorized access. The authentication process is seamless and reliable, improving overall system security. Another important enhancement is the integration of third-party APIs. Email APIs are used to send notifications related to job applications, registration, and system updates. This ensures real-time communication between the system and users. Additionally, chatbot integration is implemented to provide instant assistance to users. The chatbot helps answer frequently asked questions, guides users through the platform, and improves overall user engagement. The system also includes map integration, which allows users to view job locations visually. This feature helps job seekers understand the geographical context of job postings, making it easier to evaluate distance, accessibility, and suit-

ability. It enhances decision-making and improves the usability of the platform. Security is further strengthened in this module by implementing JWT-based authentication and role-based access control. These mechanisms ensure that different users (such as job seekers and admins) have appropriate access permissions. Sensitive data is protected using encryption and secure communication methods. Performance optimization is also addressed in this module. Techniques such as database indexing, optimized queries, and pagination are used to improve system efficiency and handle large volumes of data. These enhancements ensure faster response times and a smooth user experience even under heavy load conditions. Extensive testing is carried out to verify the integration of all advanced features. Each component is tested individually and as part of the complete system to ensure reliability and proper functionality. Overall, Module 3 transforms the Job Portal system into a feature-rich, secure, and user-friendly application by integrating modern technologies and advanced functionalities, making it suitable for real-world deployment.This module significantly improves the overall quality and usability of the system. It also provides a strong foundation for future enhancements such as AI-based recommendations and advanced analytics.These enhancements make the system more intelligent, secure, and aligned with real-world application requirements.

Advantages of this Project

- Secure authentication using Firebase and OTP verification
- User-friendly interface with easy navigation and interaction
- Efficient job search and application management system
- Integration of chatbot for instant user support
- Map-based job location visualization for better decision-making
- Real-time notifications using third-party APIs
- Scalable architecture supporting large number of users
- Strong security using JWT and role-based access control
- Optimized database performance with indexing and pagination
- Modular design allowing easy maintenance and future enhancements
- Provides centralized platform for job seekers and employers
- Reduces manual effort in recruitment process
- Ensures fast and accurate data processing
- Supports cross-platform access through web technologies
- Enhances user engagement with interactive features

a) Secure Authentication:

The system uses Firebase Authentication with OTP verification to ensure secure user login and prevent unauthorized access.

b) User-Friendly Features:

The integration of chatbot and map services provides better user interaction, guidance, and easy navigation within the platform.

c) Scalable and Efficient:

The system is designed with optimized APIs and database structure,ensuring high performance and the ability to handle large data.

Disadvantages

- Requires stable internet connection for proper functioning
- Dependence on third-party services like Firebase and APIs
- Backend-focused project with limited frontend development
- Initial setup and integration complexity is high
- Performance may decrease with very large data if not optimized
- Security risks if APIs are not properly protected
- Maintenance required for external API changes or updates
- Limited offline functionality
- Requires proper server configuration for deployment

Chapter 5

RESULTS AND DISCUSSIONS

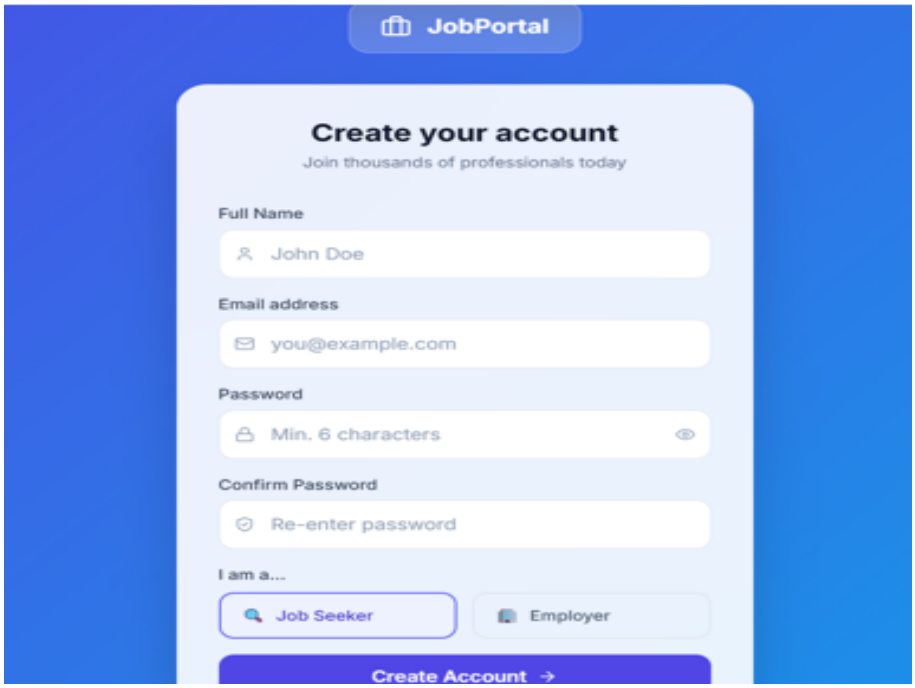

Figure 5.1: User Registration Page

Figure 5.1 illustrates The registration interface allows new users to create an account by entering personal details such as full name, email address, password, and user role (Job Seeker or Employer). The form includes proper validation mechanisms to ensure correct data entry and provides a user-friendly design.

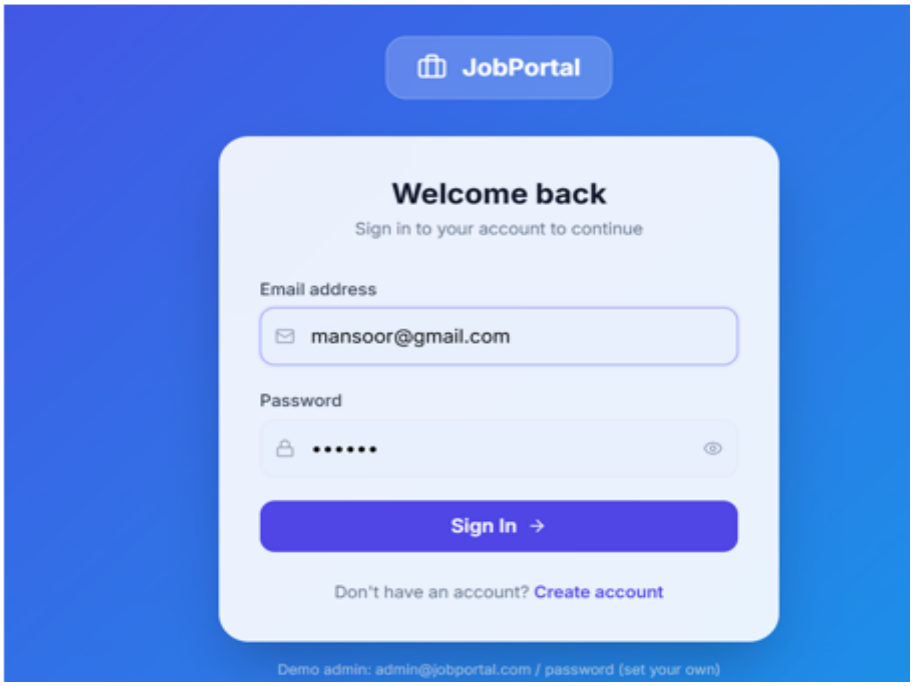

Figure 5.2: User Login Page

Figure 5.2 illustrates The login page enables registered users to securely access the system by providing their email and password. Firebase Authentication ensures secure login functionality and supports OTP-based verification. Proper error handling mechanisms are implemented to guide users in case of incorrect credentials.Error handling mechanisms are implemented to provide appropriate feedback in case of invalid credentials or failed login attempts. The page also includes navigation options for users who do not yet have an account, directing them to the registration page.

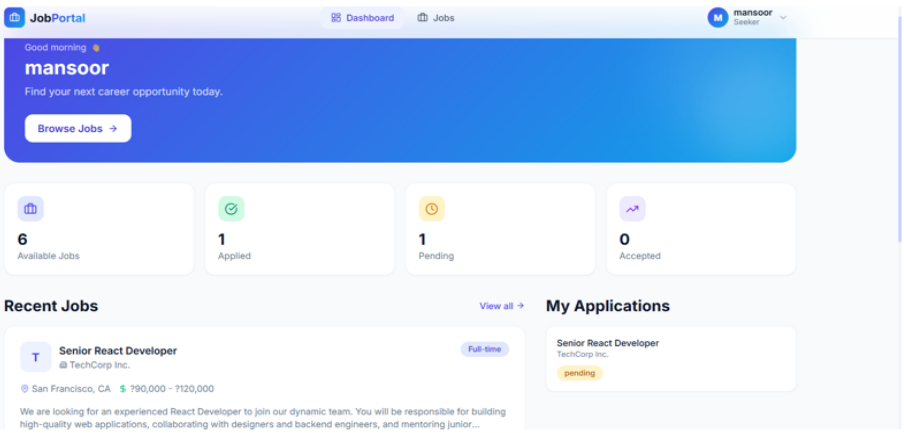

Figure 5.3: User Dashboard

Figure 5.3 illustrates The dashboard serves as the central interface for users after successful login, providing a comprehensive overview of their activity within the system. It displays key metrics such as the total number of available jobs, jobs applied for, pending applications, and accepted applications. These statistics are presented in a visually organized format, allowing users to quickly understand their current status. In addition to summary metrics, the dashboard also includes sections for recent job postings and user applications. This enables users to stay updated with the latest opportunities and track their progress efficiently. The interface is designed to be responsive and user-friendly, ensuring smooth navigation and accessibility across devices. The dashboard enhances user engagement by presenting relevant information in a structured and easily understandable manner.

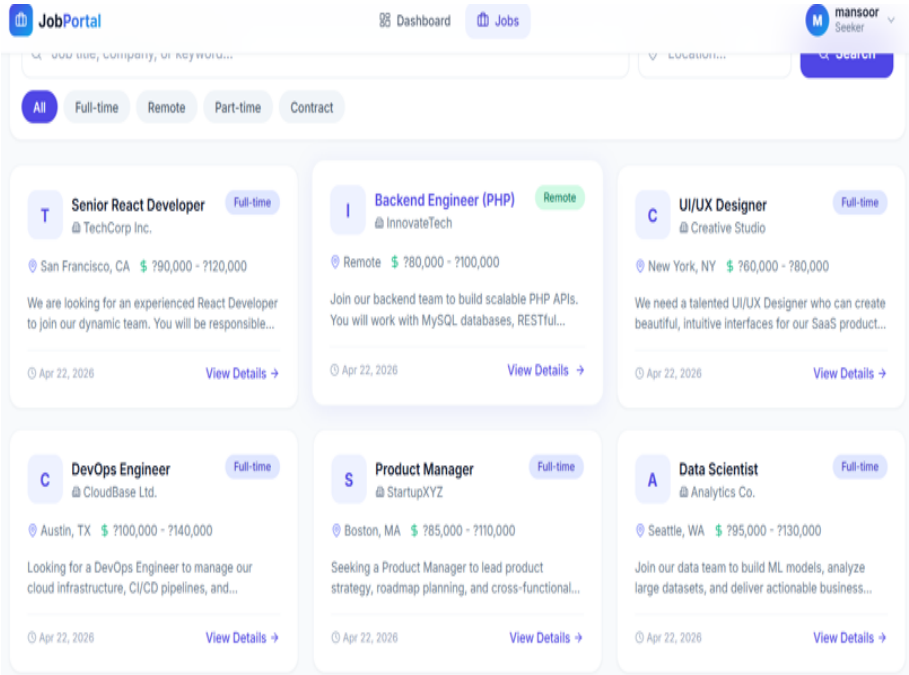

Figure 5.4: Job Listings Page

Figure 5.4 illustrates The job listings page provides a detailed view of available job opportunities within the system. Each job entry includes important information such as job title, company name, location, salary range, job type (full-time, part-time, remote), and a brief description of the role. The page also offers filtering options, allowing users to refine their search based on specific criteria such as job type or location. Users can view complete job details and apply directly through the platform, making the process seamless and efficient. The layout is designed to display multiple job postings in a structured format, ensuring clarity and readability. The inclusion of action buttons such as “View Details” improves user interaction and navigation.

The Job Portal API system was successfully developed and tested using a lightweight and modular architecture. The system integrates a frontend interface, a Core PHP-based backend, and a MySQL database, ensuring efficient communication between all components. The implementation of RESTful APIs enabled smooth data exchange using JSON format, making the system compatible with various platforms and easy to extend. The results obtained from the system demonstrate that all core functionalities are working as expected. Users are able to register and log in securely through Firebase Authentication, which provides OTP-based verification and ensures safe user identity management. This significantly enhances the security of the system compared to traditional authentication methods. The frontend interface was tested for usability and responsiveness. It provides a clean and user-friendly environment where users can easily navigate through different sections such as registration, login, dashboard, and job listings. The dashboard effectively displays user-specific data such as available jobs, applied jobs, and application status, improving user engagement and interaction. The backend system successfully handles various operations such as user authentication, job posting, job searching, and application management. The use of Core PHP allowed full control over request handling, routing, and business logic implementation. Database operations were performed using PDO, which ensures

secure execution of queries and prevents SQL injection through prepared statements. The API endpoints were thoroughly tested using Postman to verify proper request-response behavior. All HTTP methods including GET, POST, PUT, and DELETE were validated, and appropriate HTTP status codes were returned for each operation. The system demonstrated reliable performance with accurate data retrieval and storage. Error handling and validation mechanisms were implemented to ensure smooth system execution. Invalid inputs were properly detected, and meaningful error messages were returned to the user. This improves the robustness and reliability of the application. Despite the successful implementation, certain limitations were observed. The use of Core PHP without a full-stack framework may make the system less scalable and harder to maintain for large-scale applications. Additionally, dependency on external services such as Firebase may affect system performance if connectivity issues occur. Overall, the developed Job Portal system achieved its intended objectives by providing a secure, efficient, and user-friendly platform for job seekers and employers. The results indicate that the system is suitable for real-world applications, with potential for further enhancement through scalability improvements and additional features.

Chapter 6

CONCLUSION AND FUTURE ENHANCEMENTS

6.1 Conclusion

The Job Portal API system was successfully designed and implemented using a lightweight and modular architecture based on Core PHP, RESTful APIs, and MySQL database. The project effectively demonstrates the integration of frontend, backend, and database components to build a functional and user-friendly web application. The use of Firebase Authentication enhanced the security of the system by providing OTP-based verification and reliable user identity management. The backend, developed using Core PHP, handled routing, request processing, and business logic efficiently without relying on a full-stack framework. The RESTful API design ensured smooth communication between the frontend and backend using standard HTTP methods and JSON format. The system successfully supports key

functionalities such as user registration, login, job posting, job searching, and application management. The use of PDO for database interaction ensured secure query execution and protection against SQL injection attacks. Testing with Postman confirmed that all API endpoints function correctly and return appropriate responses. Overall, the project achieved its objective of developing a secure, efficient, and scalable job portal system while also strengthening the understanding of backend development concepts and API design.

6.2 Future Enhancements

Although the system performs effectively, several enhancements can be implemented to improve its scalability, performance, and usability:

- **Framework Integration:** The backend can be migrated to a full-stack framework such as Laravel to improve maintainability, scalability, and development speed.
- **Advanced Search and Filtering:** Implement AI-based job recommendations and advanced filtering options based on user preferences and behavior.
- **Real-Time Notifications:** Integrate real-time notification systems using WebSockets or Firebase Cloud Messaging for instant updates on job applications and alerts.

- **Role-Based Access Control:** Enhance security by implementing fine-grained access control for different user roles such as admin, employer, and job seeker.
- **Performance Optimization:** Introduce caching mechanisms (e.g., Redis) and optimize database queries for better performance in large-scale deployments.
- **Mobile Application Development:** Extend the system to Android and iOS platforms to improve accessibility and user reach.
- **Analytics Dashboard:** Add analytics features for employers to track job postings, application trends, and user engagement.
- **Cloud Deployment:** Deploy the system on cloud platforms such as AWS or Google Cloud for better scalability, reliability, and availability.

Chapter 7

ADDRESSING COMPLEX ENGINEERING PROBLEM AND SDG

7.1 Mapping of the Project to Complex Engineering Problem (CEP)

Table 7.1: Mapping of Project to Complex Engineering Problem (CEP)

Level	**Attribute**	**Characteristics**	**WKs**	**Justification**
WP1	Depth of Knowledge	Requires in-depth engineering knowledge	WK3, WK4, WK5, WK6	Requires knowledge of Spring Boot, Spring MVC, Spring Security, JPA, and database design for building a multi-role job portal system
WP2	Conflicting Requirements	Technical and non-technical conflicts	WK4, WK5, WK6	Balances role-based access control with usability, ensuring employers and job seekers have separate secure and intuitive workflows
WP3	Depth of Analysis	Analytical and Design thinking	WK3, WK4, WK5	It involves an entity-relationship modeling, REST API design, and integration of file upload and email notification services
WP4	Familiarity	Partially new problems	WK4, WK8	Job-matching and application tracking introduce modern challenges in multi-role web systems beyond traditional CRUD applications
WP5	Applicable Codes	Limited standard solutions	WK6	Requires custom design for resume storage, application status tracking, and dynamic job filtering not fully covered by standard frameworks

Level	Attribute	Characteristics	WKs	Justification
WP6	Stakeholder Needs	Multiple stakeholders involved	WK5, WK6, WK7	Handles distinct requirements of job seekers, employers, and admins, ensuring secure and efficient role-specific interactions
WP7	Interdependence	System-level integration	WK3, WK5, WK6	Integrates frontend (Thymeleaf), backend (Spring Boot), database (JPA/MySQL), file storage, security, and email notification into one unified system

Table 7.1 illustrates that the project maps effectively to Complex Engineering Problem (CEP) attributes by demonstrating depth of knowledge, handling conflicting requirements, and applying analytical design thinking, while also addressing partially new challenges, custom solutions, and multiple stakeholder needs. It further highlights the system's interdependence through the integration of frontend, backend, database, security, and supporting services, ensuring a comprehensive and scalable job portal solution.

7.2 Mapping of the Project to Complex Engineering Activities (CEA)

Table 7.2: Mapping of Project to Complex Engineering Activities (CEA)

EA No.	**Attribute**	**Characteristics**	**Justification based on the Project**
EA1	Range of Resources	Use of diverse technologies	The project uses Spring Boot, Spring MVC, Spring Data JPA, MySQL, Thymeleaf, Spring Security, and JavaMailSender for full-stack portal development
EA2	Level of Interactions	Complex interactions between modules	The system manages layered interactions between job listings, applications, user authentication, resume uploads, and email notifications
EA3	Innovation	Application of modern technologies	Integration of role-based dashboards, dynamic job filtering with JavaScript/ES6, file upload handling via MultipartFile, and email status notifications demonstrates innovation
EA4	Consequences to Society and Environment	Impact on users and society	Improves employment accessibility, hiring overhead, and provides a transparent and secure digital platform for job seekers and employers
EA5	Familiarity	Beyond standard practices	Combines multi-role authentication, resume management, application tracking,unified portal beyond traditional job.

The table 7.2 illustrates Mapping of Project to Complex Engineering Activities (CEA).That the project satisfies key Engineering Activities (EA) attributes by utilizing a wide range of modern technologies, enabling complex interactions between system modules, and incorporating innovative features such as role-based dashboards, dynamic job filtering, file upload handling, and email notifications. It also highlights the project's positive impact on society by improving employment accessibility and providing a secure platform.

7.3 SDG Mapping (CEP Section)

Table 7.3: SDG Mapping for the Project

SDG No.	SDG Title	Relevance to the Project
SDG 8	Decent Work and Economic Growth	Facilitates employment by connecting job seekers with employers, supporting fair recruitment processes and improving career opportunities
SDG 9	Industry, Innovation and Infrastructure	Promotes digital transformation in recruitment using Spring Boot REST APIs, dynamic filtering, and automated notification systems
SDG 10	Reduced Inequalities	Provides a standardised, accessible digital platform for all job seekers regardless of geography, reducing barriers to employment opportunities
SDG 16	Peace, Justice and Strong Institutions	Ensures data security and role-based access control using Spring Security and password hashing, maintaining integrity and fairness in the hiring process

The table 7.3 illustrates SDG Mapping for the Projec.That the project aligns with multiple Sustainable Development Goals (SDGs) by promoting decent work and economic growth through efficient job matching, supporting innovation and infrastructure with modern digital technologies, reducing inequalities by providing an accessible platform for all users, and ensuring security and fairness in the recruitment process through robust authentication and data protection mechanisms.

References

[1] Naukri.com. Available at: https://www.naukri.com

Review: One of the leading job portals in India offering a wide range of job listings, resume services, and recruiter connections. However, the interface can be complex for new users.

[2] LinkedIn Jobs. Available at: https://www.linkedin.com/jobs

Review: Provides professional networking along with job opportunities. Strong recommendation system but requires a complete profile for better results.

[3] Indeed. Available at: https://www.indeed.com

Review: A global job search platform with a simple interface and extensive listings. Lacks advanced filtering in some cases.

[4] Glassdoor. Available at: https://www.glassdoor.com

Review: Offers company reviews, salary insights, and job listings. Useful for research but some data may not always be up-to-date.

[5] Monster Jobs. Available at: https://www.monster.com

Review: One of the earliest job portals with resume and career advice services. However, it is less popular compared to newer platforms.

[6] Firebase Authentication Documentation. Available at: https://firebase.google.com/docs/auth

Review: Provides secure and scalable authentication services including OTP verification, which simplifies backend implementation.

[7] PHP Official Documentation. Available at: https://www.php.net/docs.php

Review: Comprehensive guide for Core PHP development used in building backend logic and RESTful APIs.

[8] PHP Data Objects (PDO). Available at: https://www.php.net/manual/en/book.pdo.php

Review: Provides a secure and consistent interface for database operations, helping prevent SQL injection.

[9] W3Schools PHP Tutorial. Available at: https://www.w3schools.com/php/

Review: Beginner-friendly resource for learning PHP concepts with examples.

[10] MySQL Documentation. Available at: https://dev.mysql.com/doc/

Review: Official documentation for MySQL database management system, widely used for backend storage.

[11] Bootstrap Documentation. Available at: https://getbootstrap.com/docs/

Review: Helps in designing responsive and user-friendly web interfaces efficiently.

[12] JavaScript MDN Docs. Available at: https://developer.mozilla.org/en-US/docs/Web/JavaScript

Review: Detailed and reliable resource for frontend scripting and dynamic web functionality.

[13] HTML CSS Documentation. Available at: https://developer.mozilla.org/en-US/docs/Web

Review: Core technologies for structuring and styling web pages.

[14] REST API Tutorial. Available at: https://restfulapi.net/

Review: Explains RESTful services used for communication between frontend and backend.

[15] GitHub Documentation. Available at: https://docs.github.com

Review: Useful for version control and collaborative software development.

Source Code

1. OTP Verification (PHP)

Generate OTP and Send via Email

```
<?php
session_start();
include 'db.php';

$email = $_POST['email'];
$otp = rand(100000, 999999);

$_SESSION['otp'] = $otp;
$_SESSION['email'] = $email;

// Simulated email send (use SMTP in real case)
mail($email, "OTP Verification", "Your OTP is: $otp");

echo json_encode(["message" => "OTP sent"]);
?>
```

Verify OTP

```
<?php
session_start();

$user_otp = $_POST['otp'];

if ($user_otp == $_SESSION['otp']) {
    echo json_encode(["message" => "OTP verified"]);
```

```
} else {
    echo json_encode(["error" => "Invalid OTP"]);
}
?>
```

2. Third-Party API Integration (Send Email using cURL)

```
<?php
$url = "https://api.sendgrid.com/v3/mail/send";

$data = [
    "personalizations" => [[
        "to" => [["email" => "user@example.com"]]
    ]],
    "from" => ["email" => "admin@jobportal.com"],
    "subject" => "Job Alert",
    "content" => [[
        "type" => "text/plain",
        "value" => "New job posted!"
    ]]
];

$ch = curl_init($url);
curl_setopt($ch, CURLOPT_HTTPHEADER, [
    "Authorization: Bearer YOUR_API_KEY",
    "Content-Type: application/json"
]);
curl_setopt($ch, CURLOPT_POST, 1);
curl_setopt($ch, CURLOPT_POSTFIELDS, json_encode($data));
curl_setopt($ch, CURLOPT_RETURNTRANSFER, true);

$response = curl_exec($ch);
curl_close($ch);

echo $response;
?>
```

3. Chatbot (Simple JavaScript Rule-Based)

```
function chatbotResponse(input) {
    input = input.toLowerCase();

    if (input.includes("job")) {
        return "You can search jobs in the jobs section.";
    }
    else if (input.includes("apply")) {
        return "Click on apply button near the job.";
    }
    else {
        return "Sorry, I didn't understand.";
    }
}

// Example usage
console.log(chatbotResponse("How to apply?"));
```

4. Google Maps Integration (Frontend)

```
<!DOCTYPE html>
<html>
<head>
    <title>Job Location Map</title>
    <script src="https://maps.googleapis.com/maps/api/js?key=YOUR_API_KEY"></script>
    <script>
    function initMap() {
        var location = {lat: 9.9252, lng: 78.1198}; // Madurai

        var map = new google.maps.Map(document.getElementById("map"), {
            zoom: 10,
            center: location
        });

        var marker = new google.maps.Marker({
            position: location,
            map: map
        });
```

```
    }
    </script>
</head>

<body onload="initMap()">
    <h3>Job Location</h3>
    <div id="map" style="height:400px;width:100%;"></div>
</body>
</html>
```

5. JWT Authentication (Security Enhancement)

```
<?php
require 'vendor/autoload.php';
use Firebase\JWT\JWT;

$key = "SECRET_KEY";

$payload = [
    "user_id" => 1,
    "email" => "user@example.com",
    "iat" => time(),
    "exp" => time() + 3600
];

$jwt = JWT::encode($payload, $key, 'HS256');

echo json_encode(["token" => $jwt]);
?>
```